Laborator 09 - Regression, System & Fuzz Testing

Regression testing

Testarea prin regresie este un tip de testare folosit pentru a determina dacă o bucată de cod nouă va afecta negativ alte părți ale codului pe care nu le modifică în mod direct. În mod normal, atunci când introducem cod nou într-un modul al unui proiect, există teste unitare care verifică buna funcționare a modulului respectiv. Slăbicunea acestei metode este că nu verifică și posibilele efecte laterale ale noului modul. Astfel, testarea prin regresie este caracterizată prin testarea noului modul în contextul întregului proiect.

Testare prin regresie poate fi făcută în 3 moduri:

1. Retest all
2. Regression test selection
3. Prioritization of test cases

Retest all

Cea mai costisitoare metodă, implică rerularea tuturor testelor aferente proiectului. Este de dorit să evităm această metodă pe cât posibil deoarece cosnumă foarte mult timp și foarte multe resurse resurse.

Regression test selection

Implică împărțirea testelor deja existente în suită în 2 categorii: cele care pot fi rulate pentru regresie și cele care nu sunt necesare pentru testarea prin regresie.

Prioritization of test cases

Testele sunt proritizate după impactul pecare îl are componenta testată în proiect. Componentele ale căror funcționalități sunt folosite des și/sau sunt critice sistemului vor avea o prioritate mai mare.

Pentru acest laborator vom folosi un plugin de la pytest, numit pytest-regtest, care adaugă funcționalități pentru testare de regresie.

```
$pip install pytest-regtest
```

Pentru a folosi pluginul, trebuie să oferim testelor noastre variabila expusă de pytest-regtest, numită “regtest”. Ea va fi folosită pentru a caputra output-ul testelor și a le compara la fiecare iterație

```
import pytest

def test_my_function(regtest):

    result = [i*i for i in range(10)]

    print(result, file=regtest)

    regtest.write("done")

    with regtest:
        print("this will be recorded")
```

Acest test, rulat prima dată, va eșua pentru că nu exista un fișier regtest înainte de rulare cu care să compare output-ul. Testul va eșua până când rulăm comanda

```
$py.test --regtest-reset
```

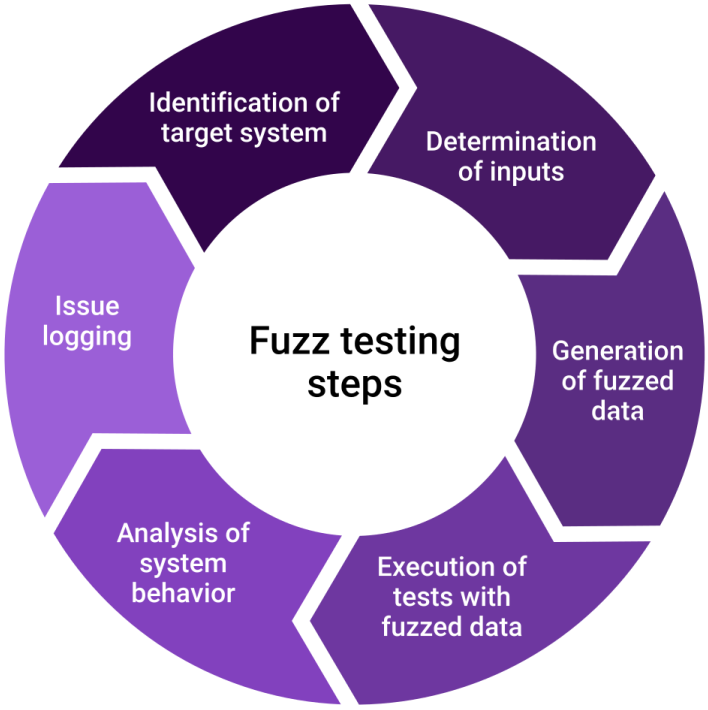
Aceasta comandă ne lasă să resetăm outputul de referință al proiectului nostru, pentru cazul în care referința anterioară nu mai este reprezentativă pentru ceea ce definim drept “bună funcționare”.

System testing

Dacă în ultimul laborator am discutat despre cum testele de integrare sunt folosite pentru a garanta că două componente interacționează corespunzător, în acest laborator vom discuta despre testarea întregului sistem. De regulă, testarea de sistem se face doar atunci când toate modulele sunt complete și integrate corespunzător, deoarece implică testarea cap-coadă a tuturor funcționalităților proiectului. Bineînțeles, acest tip de testare este necesar și pentru a verifica dacă proiectul final poate fi compilat cu succes.

Fuzz testing

Fuzz testing este o metodă de testare automată în care introducem în program input-uri invalide sau neașteptate pentru a monitoriza comportamentul acestuia. Acest tip de testare este predominant folosit pentru a testa securitatea unui sistem critic, prin a simula input-urile neobișnuite care ar putea duce la o breșă de securitate sau la brick-uirea sistemului.



De regulă, fuzzing-ul este făcut de către un fuzzer, care este un program extern ce generează input-urile eronate de care avem nevoie. Primul instinct atunci când auzim ce face un fuzzer ar putea fi să ne întrebăm "De ce am avea nevoie de un generator extern de input cand noi deja știm ce vrem de la cod?". Să luăm drept exemplu o funcție foarte simplă care calculează media:

```
def compute_avg(data):
    return sum(data) / len(data)
```

Deși este un cod incredibil de simplu, el poate genera 4 erori diferite:

1. Dacă data nu este iterabil, sum va genera TypeError
2. Dacă data nu este o colecție, len va genera TypeError
3. Dacă data are lungime 0, diviza va genera ZeroDivisionError
4. Dacă data are prea multe elemente, len va genera OverfloError

Așadar, răspunsul la întrebarea "De ce am genera input automat" este "Pentru că există prea multe tipuri de input greșit"

Pentru acest laborator vom folosi fuzzer-ul atheris.

```
$pip install atheris
```

Example code:

```
import atheris
import sys

@atheris.instrument_func
```

```
def TestOneInput(data):
    # Need to prefix the string with 'b' because atheris
    # only works with bytes and not Unicode
    if data == b"bad":
        raise RuntimeError("VERY BAD!!!")

atheris.Setup(sys.argv, TestOneInput)
atheris.Fuzz()
```

Output:

```
 === Uncaught Python exception: ===
RuntimeError: VERY BAD!!!
Traceback (most recent call last):
  File "/home/iso/An4/IC/Lab9/test.py", line 5, in TestOneInput
    def TestOneInput(data):
RuntimeError: VERY BAD!!!

==8793== ERROR: libFuzzer: fuzz target exited
SUMMARY: libFuzzer: fuzz target exited
MS: 4 InsertByte-CopyPart-ShuffleBytes-CMP- DE: "bad"-; base unit: adc83b19e793491b1c6ea0fd8b46cd9f32e592fc
0x62,0x61,0x64,
bad
artifact_prefix='./'; Test unit written to ./crash-1902e3d6fc4e78a0bcc50ba12b882769afbf4a8c
Base64: YmFk
```

În același fișier în care se află executabilul atheris va genera un nou fișier text cu numele hash-ului generat, în care va scrie input-urile care au dus la generarea erorilor. În cazul de mai sus va crea dun fișier care conține doar cuvântul “bad”

Pentru a-i putea spune lui atheris care părți din cod trebuie instrumentate. Pentru a face acest lucru avem 3 opțiuni:

Pentru a instrumenta o anumită funcție:

```
with atheris.instrument_imports():
  import some_lib
  from another_lib import random_feature
```

Pentru a instrumenta biblioteci importate:

```
@atheris.instrument_func
def my_func(my_input):
  print("instrumented")
```

Pentru a instrumenta tot ce poate fi instrumentat:

```
atheris.instrument_all()
atheris.Setup()
```

Exerciții

0) Citiți documentația pentru atheris: https://github.com/google/atheris [https://github.com/google/atheris]

1) Pentru început, scrieți o colecție de 5 funcții care pot genera excepții în cazul datelor de intrare eronate și scrieți teste pentru fiecare funcție. Folosiți atheris pentru a depista inputurile care ar putea genera erori

2) Pentru aceleași funcții de mai sus, creați încă un set de teste de regresie, generați primul output de referință, apoi alterați testele și analizați mesajele de eroare.

tsc/laboratoare/laborator-09.txt · Last modified: 2024/05/14 23:51 by dragos.isopencu